# WBCS040-05
# Information Retrieval

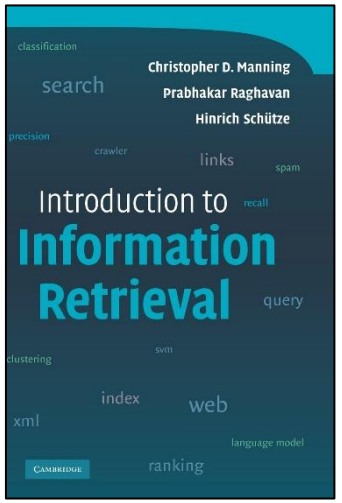
# Relevance Feedback and Query Expansion

**University of Groningen**
**2026–2027**

**Prof. Owen Falzon**
**Centre for Biomedical Cybernetics**
**University of Malta**

**owen.falzon@um.edu.mt**

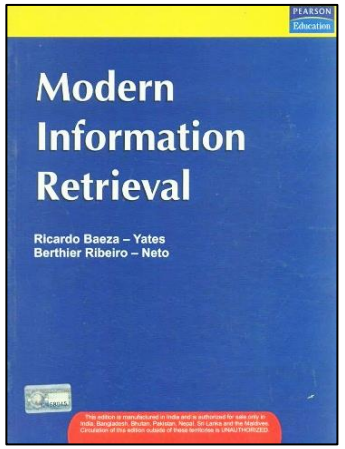
# Reference Textbooks




**Introduction to Information Retrieval**

Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze

*Chapter 9 - Relevance Feedback & Query Expansion*




**Modern Information Retrieval**

Ricardo Baeza-Yates and Berthier Ribeiro-Neto

*Chapter 5 - Relevance Feedback & Query Expansion*

# 1. Introduction to Relevance Feedback

# Relevance Feedback: Query - Document Mismatch

- Central challenge in information retrieval (IR):
  - **The retrieval system sees the query, not the user's full information need.**
  - A query is often a short and incomplete representation of the user's underlying information need.

    e.g., *"I want to know how to fix water damage caused by a leaking pipeline"* - abbreviated to *"pipeline leaks"*.
  - This mismatch is the driving force behind **relevance feedback** and **query expansion**
    - attempt to bridge the gap between user intent and system representation.

- **Information Need**: the underlying goal that motivates the search.
  - Example: *"I want to learn how cracks form in water pipes and how to repair them to stop leaks."*

- **Query**: the explicit string of terms entered into the search box, and that the system sees.
  - Example: *"pipe leaks"*.

- This **problem remains in modern AI search**: even an LLM-based system must infer the user's intended information need from the query and available context.

# Relevance Feedback: Query - Document Mismatch

- Relevance defines whether a document in the collection is useful for satisfying a user's information need.

- Let:
  - $Q$ = set of all possible queries.
  - $D$ = set of all documents in the collection.
  - $q \in Q$: one particular query
  - $d \in D$: one particular document
  - $Q \times D$ : all possible query–document pairs

- **Relevance** is a **binary relation**, linking two sets (queries and documents) via ordered pairs,

$$R \subseteq Q \times D \qquad \text{set of query–document pairs judged relevant}$$

- If $(q, d) \in R$, document $d$ is considered **relevant** to query $q$.
- This formulation treats relevance as a property linking queries to documents.

***** Sidenote: **binary relation** means a relation between two sets;
it is different from **binary relevance**, where relevance has only two values: relevant / non-relevant.

# Relevance: Query - Document Mismatch

- From a technical perspective, we can think of the information need as defining an **ideal set of documents** in the collection
  - This set $D^*$contains every document that would be relevant to the user's true intent.

$$D^*(q) \subseteq D \text{ = subset of documents relevant to a particular query } q.$$

$$D^*(q) = \{d \in D : \ d \text{ satisfies the user's information need}$$

  - In practice, the system does not observe the information need directly - it only observes the query $q$.
  - The system can only **approximate** $D^*$ through the words the user provides
    - often leads to mismatches between what the user really wants and what the system retrieves.

# Relevance Feedback

- **Relevance feedback (RF)**
  - Users may not be able to fully articulate their information needs in the form of an initial query.
  - However, they are usually able to recognise relevant documents when they see them.
  - This feedback can be exploited through an **interactive loop** between the user and the retrieval system to produce improved rankings in subsequent iterations.

- **Relevance feedback loop:**

**1. User submits query, $q$**
  - Typically short, keyword-based, and potentially ambiguous.
  - E.g. A user searching for “space applications”.

**2. System retrieves initial ranked list**
  - Documents are ranked using the current retrieval model (e.g., vector space, BM25).
  - The ranking reflects the system’s initial guess of relevance.

**3. User provides judgements**

- A subset of results is labelled:

  **Relevant documents,** $D_r$, judged useful by the user.

  **Non-relevant documents,** $D_{nr}$, judged not useful.

**4. System updates query representation**

- The system adjusts term weights or the query model itself.
- Uses information from $D_r$ and $D_{nr}$ to refine its understanding of what is relevant.

**5. New ranking is produced**

- A revised document ordering is generated.
- Ideally, more relevant documents move toward the top of the ranking.

# Relevance and Retrieval in Modern LLM Systems: RAG

- An LLM does not necessarily contain the latest, specialised or private information needed to answer a query.

- **Retrieval-Augmented Generation (RAG)** combines an information retrieval system with a Large Language Model (LLM).

Retrieve information → Augment the model's context → Generate the answer



# Relevance and Retrieval in Modern LLM Systems: RAG

- Importance of reliable retrieval and ranking:
  - Retrieving and ranking the right information is critical: relevant evidence ranked too low may never reach the model.
  - Typically triggered when external information is likely to improve the answer, for example for current, specialised, private, or source-grounded information.
  - Retrieval routing is generally used: should the system answer directly, or should it first retrieve external information?
    - This decision can be based on rules, learned classifiers, or an LLM to decide whether external evidence is needed.

# Relevance Feedback

- To retrieve and rank documents, whether in a conventional information retrieval system or in the retrieval stage of an LLM/RAG system, a system needs a way to represent the query and documents and estimate how well they match.

- A classical approach is the **Vector Space Model (VSM)**
  - queries and documents are represented as vectors and ranked according to their similarity.

- Another influential approach is the **Probabilistic Model of Relevance**
  - How likely is this document to be relevant to the query?

# 2. Vector Space Model

# Vector Space Model (VSM)

- Representing queries and documents so that relevance can be measured computationally.

  $D = \{d_1, d_2, \dots, d_N\}$ a collection of documents

  $V = \{t_1, t_2, \dots, t_M\}$ the vocabulary (index terms)

  $q = \{t_{q_1}, t_{q_2}, \dots, t_{q_M}\}$ the query posed by the user

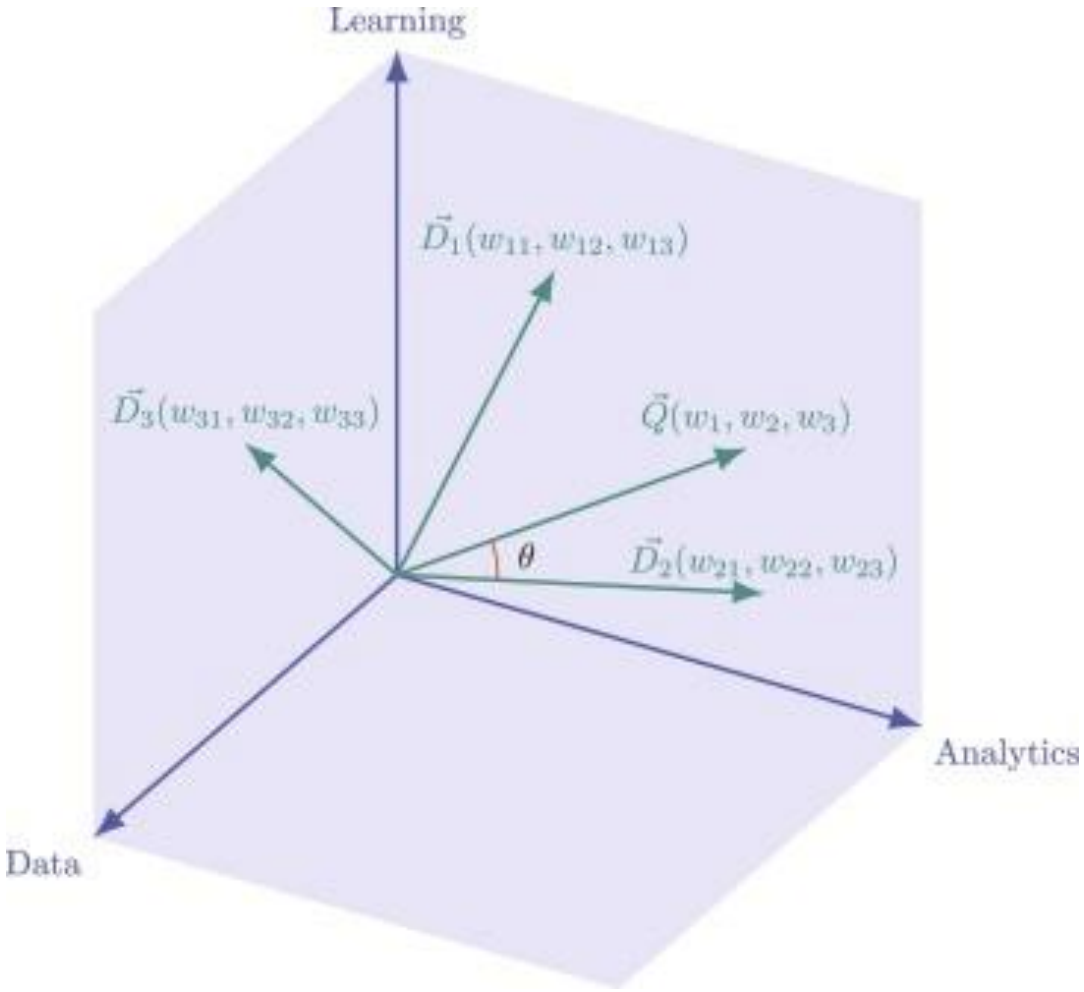
- Each document and query is embedded as a vector in $\mathbb{R}^M$ in vector space:

  $\overrightarrow{d_i} = (w_{i1}, w_{i2}, \dots, w_{iM})$ $\quad \vec{q} = (w_{q1}, w_{q2}, \dots, w_{qM})$

  $w_{ij}$ is the weight of vocabulary term $t_j$ in document $d_i$

  $w_{qj}$ is the weight of vocabulary term $t_j$ in the query $q$.

  - Both documents and queries share the **same vector space representation**.
  - Generally, the terms in the original query are initially assigned a value of 1.
  - The document weights, $w$, may then be updated through different schemes:
    - binary (presence/absence)
    - term frequency (TF)
    - more sophisticated schemes such as term frequency - inverse document frequency (TF-IDF).




**Three-dimensional vector space example**

**Adapted from: https://www.sciencedirect.com/topics/computer-science/vector-space-models**

# Vector Space Model (VSM)

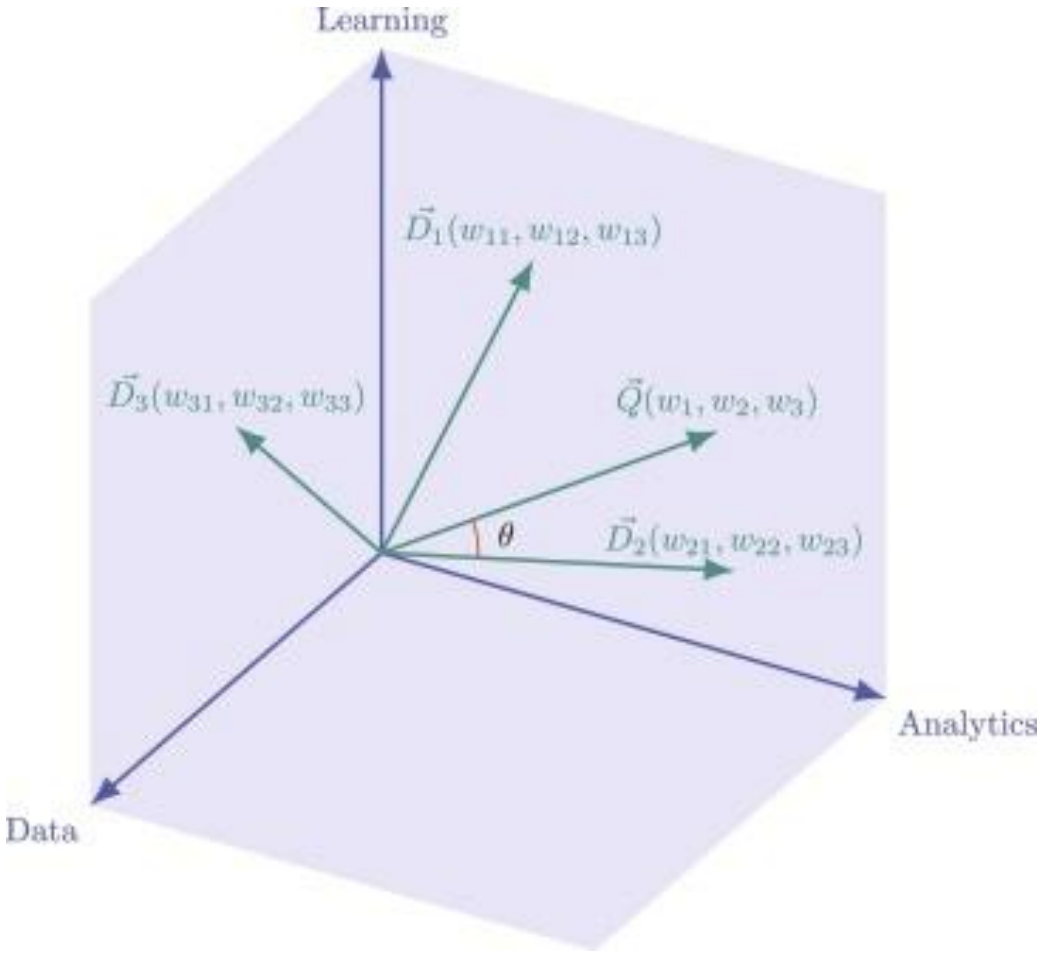
- Similarity between documents and queries is measured with **cosine similarity**:

$$\text{sim}\left(\vec{q}, \overrightarrow{d_i}\right) = \frac{\vec{q} \cdot \overrightarrow{d_i}}{\| \vec{q} \| \| \overrightarrow{d_i} \|}$$

- The system outputs a ranking $R(q) = \langle d_{i_1}, d_{i_2}, \ldots, d_{i_k} \rangle$, ordered by decreasing similarity.

- The assumption here is that the query vector $\vec{q}$ is a close representation of the true information that is needed. In practice, this assumption rarely holds.

- **Geometric Interpretation**
  - Each dimension represents a vocabulary term.
  - A query is a vector in the same space as docs.
  - The retrieval process involves finding closest documents (vector with the most similar orientation) to $\vec{q}$.




**Three-dimensional vector space example**

**Adapted from: https://www.sciencedirect.com/topics/computer-science/vector-space-models**

# Term Weighting for Document Scoring

| Weighting Scheme | Definition / Formula | Interpretation | Advantages | Limitations | Typical Use Cases |
|---|---|---|---|---|---|
| **Binary** | $w_{t,d} = \begin{cases} 1 & \text{if } t \in d \\ 0 & \text{otherwise} \end{cases}$ | Presence or absence only | Simple; fast; good for very short texts | Ignores frequency; weak discrimination between documents | Boolean retrieval, short fields (titles, metadata) |
| **Term Frequency (TF)** | **Raw count:** $tf(t,d) = f_{t,d}$<br>**Normalised:** $tf(t,d) = \frac{f_{t,d}}{\max_{t'} f_{t,d}}$ prevents bias toward longer documents.<br>**Log-scaled:** $tf(t,d) = 1 + \log(f_{t,d})$ diminishing returns for high counts. | Frequent terms in a document are more important | Captures repetition | Biased toward longer documents; common terms may dominate | Document ranking, text classification |

*t = term* *d = document* *N = total number of documents* $df(t)$*=document frequency (number of documents containing t)*

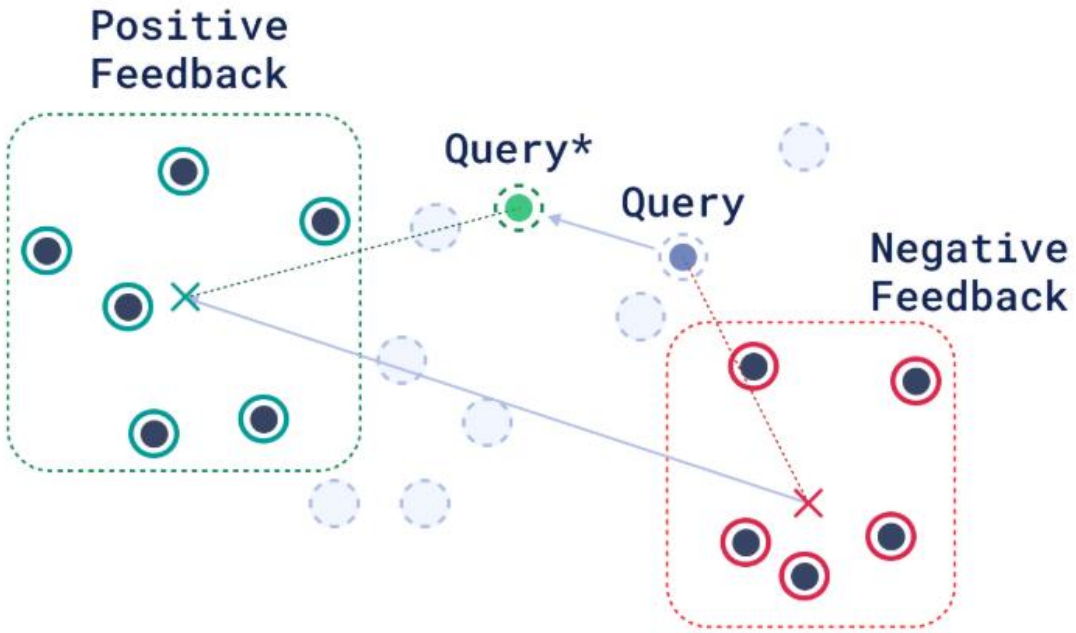
# Term Weighting for Document Scoring

| Weighting Scheme | Definition / Formula | Interpretation | Advantages | Limitations | Typical Use Cases |
|---|---|---|---|---|---|
| **Term Frequency – Inverse Document Frequency (TF–IDF)** | $w_{t,d} = tf(t,d) \times \log \frac{N}{df(t)}$ | High if frequent in a document but rare across the corpus | Balances local importance and global distinctiveness | Still heuristic; requires corpus statistics | Search engines, IR systems, keyword extraction |

*t = term* *d = document* *$N$ = total number of documents* *$df(t)$=document frequency (number of documents containing $t$)*

# Relevance Feedback in the Vector Space Model

- The **vector space model** (VSM) represents both queries and documents as vectors in a high-dimensional space, where each axis corresponds to a distinct term (word or token) in the collection vocabulary.
- The similarity between a query and a document is measured using metrics such as **cosine similarity**.
- In this setting, **relevance feedback** becomes a process of adjusting the query vector so that it points closer to relevant documents and further away from non-relevant ones.
- Each **dimension (axis)** of this space **corresponds to a word (or term)** in the vocabulary.
- The **coordinates of the vector are weights** for how important each word is in that document or query.

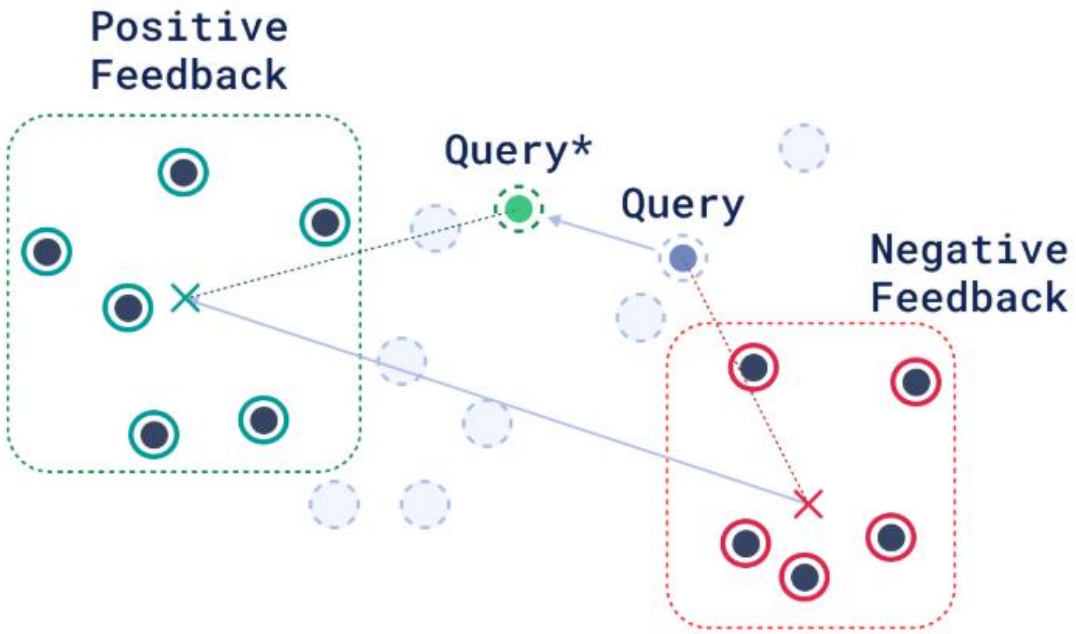



**Query Vector Updates Representation**

**Adapted from: https://qdrant.tech/articles/search-feedback-loop/**

# Relevance Feedback in the VSM: Rocchio Algorithm

- The Rocchio algorithm is a method to **improve search queries based on user feedback**.
- **The problem**: the original query is often "pointing" in the wrong direction because it only has a few words.
- Ask the user which documents are **relevant** and which are **not relevant**.
  - Move the query **towards the centroid of relevant docs** (to include their vocabulary).
  - Move the query **away from the centroid of non-relevant docs** (to avoid their vocabulary).
  - Keep part of the **original query** so it doesn't drift too far.




**Query Vector Updates Representation**

**Adapted from: https://qdrant.tech/articles/search-feedback-loop/**

# Relevance Feedback in the VSM: Rocchio Algorithm

$$\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \frac{\gamma}{|D_{NR}|}\sum_{\vec{d}\in D_{NR}}\vec{d}$$

$\overrightarrow{q_m}$: The **modified query vector** after feedback. This is the query representation used for the next retrieval iteration.

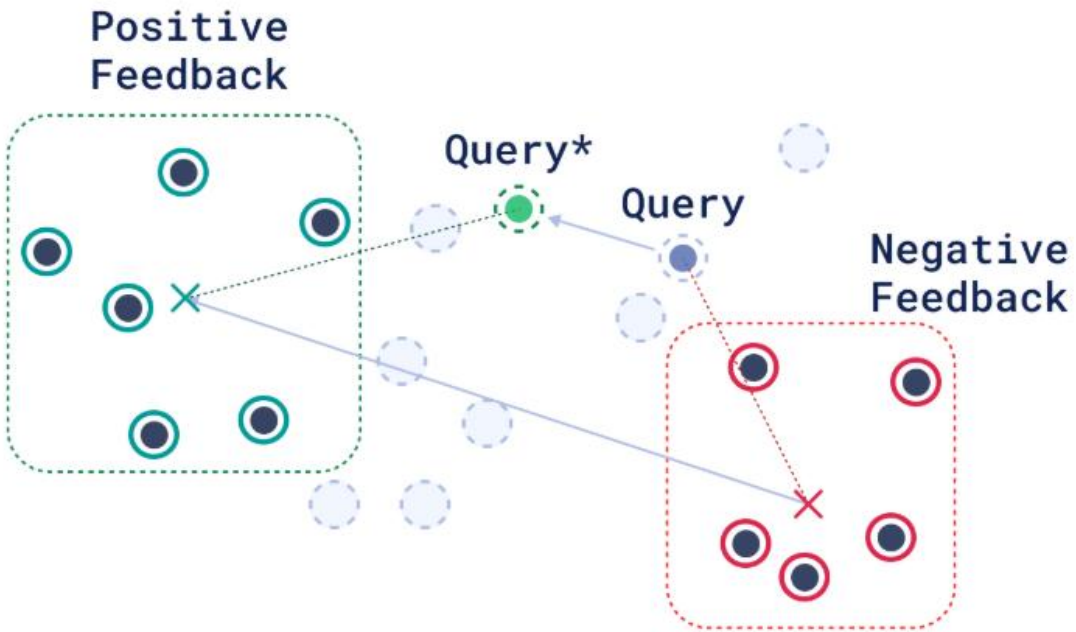
$\overrightarrow{q_0}$: The **original query vector** as first formulated by the user. Each dimension corresponds to a vocabulary term, weighted by its presence (e.g., TF, TF-IDF).

$D_R$: The set of documents explicitly marked as **relevant** by the user during feedback.

$|D_R|$: number of relevant documents in the judged set.

$D_{NR}$: The set of documents explicitly marked as **non-relevant**.

$|D_{NR}|$: number of non-relevant documents in the judged set.




**Query Vector Updates Representation**

**Adapted from: https://qdrant.tech/articles/search-feedback-loop/**

# Relevance Feedback in the VSM: Rocchio Algorithm

$$\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \frac{\gamma}{|D_{NR}|}\sum_{\vec{d}\in D_{NR}}\vec{d}$$

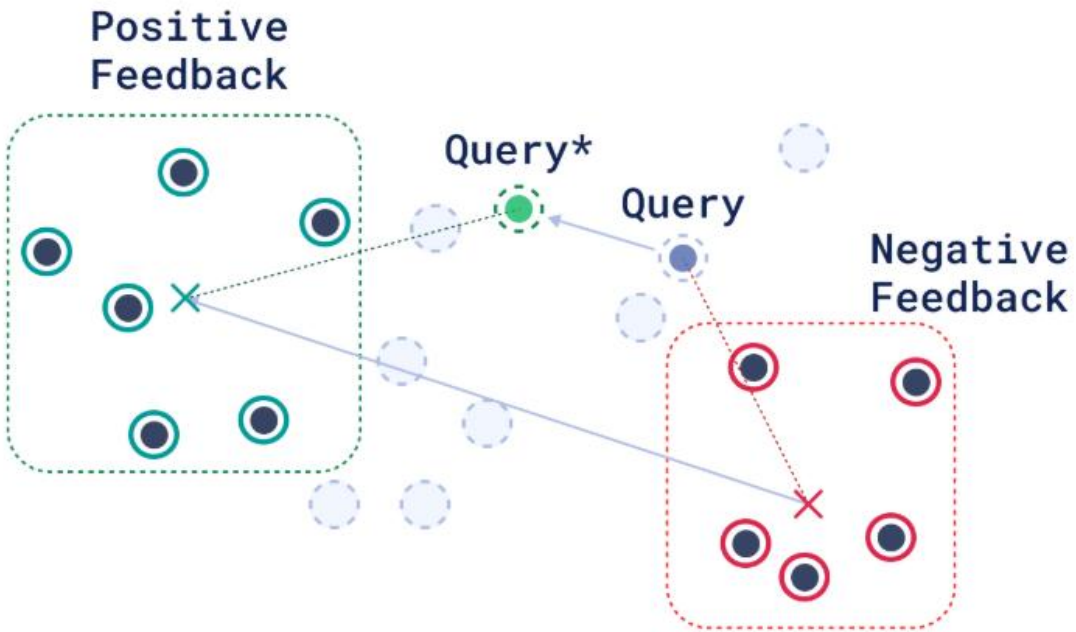
$\vec{d}$: A **document vector** in the same vector space as the query. Each document is represented as $(w_1, w_2, \ldots, w_M)$ ,with weights for each vocabulary term.

$\alpha$: A weighting parameter controlling the contribution of the **original query**. Typical value: $\alpha = 1$.

$\beta$: A weighting parameter controlling the influence of **relevant documents**. Typical value: $\beta \approx 0.75$.

$\gamma$: A weighting parameter controlling the influence of **non-relevant documents**. Typical value: $\gamma \approx 0.15$.

Often set small or even zero, since negative feedback is noisy.




**Query Vector Updates Representation**

**Adapted from: https://qdrant.tech/articles/search-feedback-loop/**

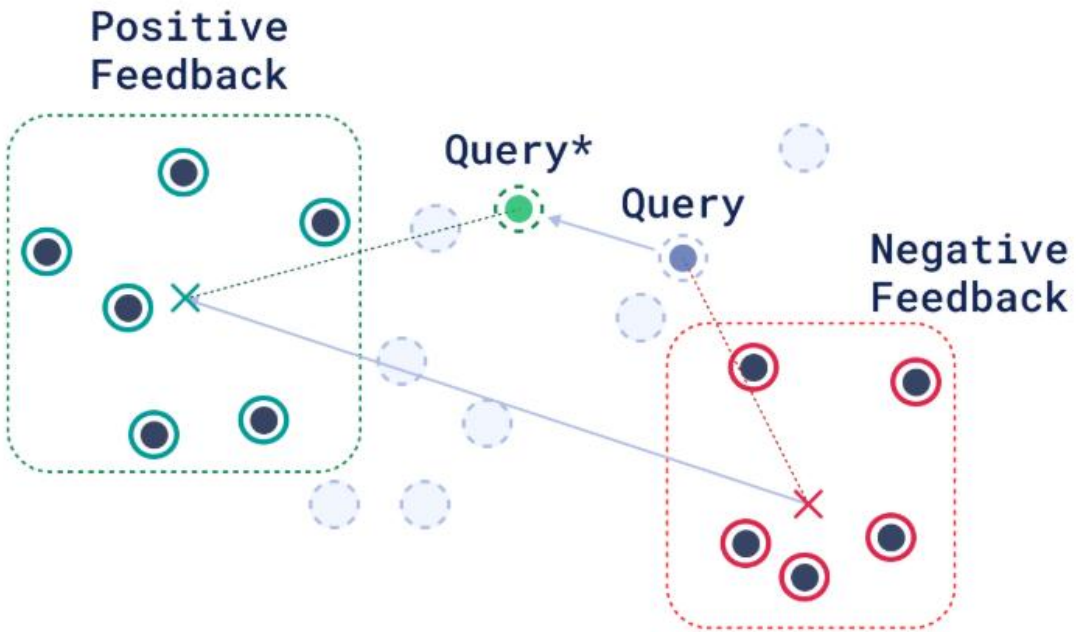
# Relevance Feedback in the VSM: Rocchio Algorithm

$$\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \frac{\gamma}{|D_{NR}|}\sum_{\vec{d}\in D_{NR}}\vec{d}$$

- The **first term**, $\alpha\overrightarrow{q_0}$, keeps the original query, so the system does not drift too far.
- The **second term,** $\frac{\beta}{|D_R|}\sum\vec{d}$, pulls the query vector toward the centroid of relevant documents.
- The **third term**, $-\frac{\gamma}{|D_{NR}|}\sum\vec{d}$, pushes the query vector away from the centroid of non-relevant documents.
- **New Query = Original Query**

  **+ Boost from Relevant Docs**

  **– Penalty from Non-Relevant Docs**




**Query Vector Updates Representation**

**Adapted from: https://qdrant.tech/articles/search-feedback-loop/**

# Relevance Feedback in the VSM: Example

- Example
  - Query: “insulin in diabetes”
  - Note: stopwords like ‘in’ are expected to occur in practically every document, and are generally removed when querying (TF-IDF would give a weight close to 0).

- Term of interest: ‘insulin’, ‘diabetes’
  - Collection: 5 documents (N=5)

| Doc/Terms | diabetes | insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |

Considering just the terms in the query, ‘diabetes’ and ‘insulin’, each document can be represented as a term vector in $\mathbb{R}^2$.

$$D1 = (1, 1)$$
$$D2 = (0, 0)$$
$$D3 = (1, 0)$$
$$D4 = (0, 1)$$
$$D5 = (1, 0)$$

Initial query vector $Q_0 = (1, 1)$

# Relevance Feedback in the VSM: Example

Initial Ranking using Cosine Similarity between the document vector and the query vector

$$sim(\mathrm{Q}, \mathrm{D}) = \frac{Q \cdot D}{\| Q \| \| D \|}$$

| | diabetes | insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |
| | | |
| **$Q_0$** | 1 | 1 |

- $sim(Q, \mathrm{D1}) = (1 \cdot 1 + 1 \cdot 1)/(\sqrt{2} \cdot \sqrt{2}) = 1$
- $sim(Q, \mathrm{D2}) = 0$
- $sim(Q, \mathrm{D3}) = (1 \cdot 1 + 1 \cdot 0)/(\sqrt{2} \cdot 1) = 0.707$
- $sim(Q, \mathrm{D4}) = (0 \cdot 1 + 1 \cdot 1)/(\sqrt{2} \cdot 1) = 0.707$
- $sim(Q, \mathrm{D5}) = (1 \cdot 1 + 1 \cdot 0)/(\sqrt{2} \cdot 1) = 0.707$

**Initial Ranking (cosine similarity with $Q_0$ = (1, 1))**
D1 matches both terms - top ranked.
D3 and D5 match *diabetes* only.
D4 matches *insulin* only.
D2 matches nothing.

Initial ranking: **D1 > D3/D4/D5 > D2**

# Relevance Feedback in the VSM: Example

Let's suppose the **user provides the following feedback** after the initial list of documents is provided.

**Relevant = {D1, D3}**
**Non-relevant = {D4, D5}**

| Doc/Terms | diabetes | insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |

**Rocchio update** $\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \frac{\gamma}{|D_{NR}|}\sum_{\vec{d}\in D_{NR}}\vec{d}$ with $\alpha = 1, \beta = 0.75, \gamma = 0.25.$

- Centroid of relevant docs $(D1 + D3)/2 = \big((1,1) + (1,0)\big)/2 = (1, 0.5)$
- Centroid of non-relevant docs $(D4 + D5)/2 = \big((0.1) + (1,0)\big)/2 = (0.5, 0.5)$

- New modified query vector:

$$Q_m = (1,1) + 0.75(1, 0.5) - 0.25(0.5, 0.5)$$
$$Q_m = (1,1) + (0.75, 0.375) - (0.125, 0.125)$$
$$Q_m = (1.625, 1.25)$$

# Relevance Feedback in the VSM: Example

**New ranking**

$Q_m = (1.625, 1.25)$

$Q_m$ = 1.6252 + 1.252 = 2.049

- $D1$: $\frac{1.625+1.25}{2.049 \cdot \sqrt{2}} = 0.993$
- $D3$: $\frac{1.625}{2.049 \cdot 1} = 0.793$
- $D5$: $\frac{1.625}{2.049 \cdot 1} = 0.793$
- $D4$: $\frac{1.25}{2.049 \cdot 1} = 0.610$
- $D2$: $0$

| | Diabetes | Insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |
| | | |
| **$Q_m$** | 1.625 | 1.25 |

**New ranking: D1 > D3/D5 > D4 > D2**

(Compared to Initial ranking: D1 > D3/D4/D5 > D2)

- Initially: D3, D4, and D5 were tied.
- After feedback:
  - Relevant doc D3 is boosted above D4.
  - Non-relevant doc D4 is pushed down.

- The feedback shifts the query vector towards the centroid of the relevant cluster and away from non-relevant ones.

# Relevance Feedback in the VSM: Example

**Considering additional terms**

- Note that in practice, in the feedback algorithm, the updated query vector incorporates all terms from judged relevant docs (including ones not in the query), so additional terms get introduced.
- In practice, this expansion is limited to a number of top terms (say 10 or 20 terms) to avoid 'bloated' queries.

| | diabetes | insulin | glucose |
|---|---|---|---|
| **D1** | 1 | 1 | 1 |
| **D2** | 0 | 0 | 0 |
| **D3** | 1 | 0 | 1 |
| **D4** | 0 | 1 | 0 |
| **D5** | 1 | 0 | 0 |
| | | | |
| **$Q_0$** | 1 | 1 | 0 |

Rocchio update $\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \frac{\gamma}{|D_{NR}|}\sum_{\vec{d}\in D_{NR}}\vec{d}$ with $\alpha = 1, \beta = 0.75, \gamma = 0.25.$

- Centroid of relevant docs $(D1 + D3)/2 = \big((1,1,1) + (1,0,1)\big)/2 = (1, 0.5, 1)$
- Centroid of non-relevant docs $(D4 + D5)/2 = \big((0,1,0) + (1,0,0)\big)/2 = (0.5, 0.5, 0)$

- New modified query vector:

$$Q_m = (1,1,0) + 0.75(1, 0.5, 1) - 0.25(0.5, 0.5, 0)$$
$$Q_m = (1,1,0) + (0.75, 0.375, 0.75) - (0.125, 0.125, 0)$$
$$Q_m = (1.625, 1.25, 0.75)$$

# Relevance Feedback in the VSM: Example

- $Q_m = (1.625, 1.25, 0.75)$

**Interpretation of $Q_m$**

- **'diabetes'**: 1.625 (most strongly boosted)
- **'insulin'**: 1.25 (moderately reinforced)
- **'glucose'**: 0.75 (new term introduced from relevant docs)

| Doc/Terms | diabetes | insulin | glucose |
|---|---|---|---|
| **D1** | 1 | 1 | 1 |
| **D2** | 0 | 0 | 0 |
| **D3** | 1 | 0 | 1 |
| **D4** | 0 | 1 | 0 |
| **D5** | 1 | 0 | 0 |
| | | | |
| **$Q_m$** | 1.625 | 1.25 | 0.75 |

- $\| Q_m \| \approx 2.184$
- $D1$: $\frac{1.625+1.25+0.75}{2.184\sqrt{3}} \approx 0.959$
- $D3$: $\frac{1.625+0+0.75}{2.184\sqrt{2}} \approx 0.769$
- $D5$: $\frac{1.625}{2.184 \cdot 1} \approx 0.745$
- $D4$: $\frac{1.25}{2.184 \cdot 1} \approx 0.573$
- $D2$: $0$

**After feedback: D1 > D3 > D5 > D4 > D2**

(Compared to Initial ranking: D1 > D3/D4/D5 > D2)

# Relevance Feedback in the VSM

## Negative Feedback is Problematic

- **Judgement difficulty**
  - It's easy for users to recognise a good (relevant) doc.
  - Harder to decide if something is "non-relevant" (is it completely irrelevant, or just partially useful).
- **Heterogeneity of non-relevant docs**
  - Non-relevant documents come from many unrelated clusters.
  - Their centroid is not meaningful and can be misleading (it points in "everywhere but relevant" direction).
- **Risk of query drift**
  - Over-penalising terms from one irrelevant doc may suppress terms that are actually useful in another relevant context.
- **User reluctance**
  - In real systems, users rarely invest effort to mark non-relevant results.
  - They are much more willing to flag positives.

# Variants of the Rocchio Algorithm

## 1. Positive-Only Feedback

- **Definition:** set $\gamma = 0$ in the Rocchio formula:

$$\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d}$$

- **Motivation**
  - Negative feedback (non-relevant docs) is often unreliable.
  - Users may not carefully judge “non-relevance”
  - In web settings, users rarely mark non-relevant docs at all.

- **Practical effect**
  - Only pulls query vector **towards relevant docs**, no “push away.”
  - Prevents query drift from spurious non-relevant labels.

- **Use cases**
  - Image retrieval, biomedical search (high recall domains - retrieving as many of the truly relevant documents as possible).
  - Web search systems that allow “find more like this” functionality.

# Variants of the Rocchio Algorithm

**2. IDE dec-hi Variant**

- Instead of averaging all non-relevant docs, use only the **single highest-ranked non-relevant document** in the negative feedback term **(dec-hi = decrement–highest weight)**

$$\overrightarrow{q_m} = \alpha\overrightarrow{q_0} + \frac{\beta}{|D_R|}\sum_{\vec{d}\in D_R}\vec{d} - \gamma\vec{d}_{NR}^{top}$$

- where $\vec{d}_{NR}^{top}$ is the highest-ranked non-relevant doc.

- **Motivation**
  - Most non-relevant docs are “uninteresting noise.”
  - The top non-relevant doc is the one most likely to “confuse” the system (ranked highly by mistake).
  - Targeting it directly provides a sharper correction.

- **Outcome**
  - Often performs better than using all non-relevant docs.
  - More stable across different collections.

- **Intuition:**
  - Corrects the system’s “worst error” instead of diluting with all negatives.

# Cluster Assumption

- Relevant documents are assumed to form a single, cohesive cluster in vector space.
- Feedback moves the query toward the centroid of this cluster, and away from nonrelevant docs.
- In practice, this assumption can fail for a number of reasons.

| Failure Case | Example | Challenge |
|---|---|---|
| **Synonymy (multiple vocabularies for same concept)** | Burma vs. Myanmar | Relevant docs split into two vocabulary clusters. Rocchio centroid falls in between, not close to either. |
| **Disjunctive or Multi-Intent Queries** | "Pop stars who once worked at Burger King"<br><br>Felines: lions, tigers, domestic cats | Relevant docs cover multiple disjoint clusters (different celebrities). Rocchio centroid averages them, pointing nowhere useful.<br><br>Relevant docs form several subclusters. Centroid compromises by over-favouring common words ("animal"), under-weighting distinctive species. |
| **Heterogeneous Nonrelevant Documents** | Query: "Apple Inc. revenue" irrelevant docs about apple fruit, apple pies, gardening | Nonrelevant docs are scattered everywhere; centroid subtraction suppresses useful overlapping terms (e.g., "apple"). |

# The Vocabulary Problem

- Users may employ words different from those in relevant documents.

**Synonymy**
- Multiple terms correspond to the same concept.
- *Example*: doctor vs. physician, pipeline rupture vs. pipeline leak.
- *Consequence*: Relevant documents may not match query terms.

**Polysemy**
- One term corresponds to multiple concepts.
- *Example*: jaguar (animal vs. car brand).
- *Consequence*: Irrelevant documents are retrieved (low precision).

**Morphological variation**
- Inflected and derived forms: operate, operates, operating, operation.
- Stemming (reduce to a common stem) or lemmatisation (map words to their dictionary form) is often necessary to collapse these forms.
- Without stemming or lemmatisation, the query and document spaces become unnecessarily sparse

**Contextual ambiguity**
- Words whose meaning depends on domain.
- Example: cell in biology vs. cell in telecommunications.

- This problem is amplified in short queries, where limited context makes disambiguation very challenging.

# Pros and Cons of the Vector Space Model

**Advantages of VSM**

- **Simple and intuitive:** Easy to understand and implement.
- **Good performance:** Often provides reasonable relevance rankings.
- **Partial matching:** Can retrieve useful documents even if these only partially match the query.
- **Term weighting:** Allows for sophisticated weighting schemes to improve relevance.

**Disadvantages of VSM:**

- **High dimensionality:** The vector space can become very large for large vocabularies, leading to computational challenges.
- **"Bag of Words" model:** It treats documents as unordered collections of words, ignoring word order and semantic relationships (e.g., "dog bites man" vs. "man bites dog").
- **Assumes term independence:** Treats terms as separate/orthogonal dimensions and does not explicitly model relationships between them.).
- **Synonymy and Polysemy:** Difficulty handling synonyms (different words, same meaning) and polysemy (same word, different meanings).

- **Vector Space Model:** *How similar is this document to the query?*
  **Probabilistic models ask:** *How likely is this document to be relevant to the query?*

# 3. Probabilistic Models of Relevance

# Probabilistic Model of Relevance

- From similarity to probability of relevance, the **Probabilistic Model of Relevance**, also called the **Binary Independence Model (BIM)**, estimates the probability that a document is relevant to a query, given the words present in both the document and the query
  - Unlike Rocchio's algorithm, relevance is not represented as a single centroid in vector space.
- Assumptions to make the problem tractable:

**1. Binary Representation**

- Both documents and queries are represented as binary vectors.
- we only care if a term is present or absent in a document/query, not how many times it appears.
- E.g., if a document contains "apple," its vector for "apple" would be 1, regardless of whether "apple" appears once or ten times.

**2. Document Term Independence**

- the probability of observing one term in a document is independent of the probability of observing another term in the same document
- E.g., knowing that "apple" is in a document doesn't say anything about whether "pie" is in the same document - often not true in reality but simplifies the math significantly.

**3. Query Term Independence**

- the model also assumes that the terms in the query are also independent of each other.

# Probabilistic Model of Relevance

- The goal is to calculate the probability that
  - a document $D$ is relevant to a query $Q$, denoted as $P(R|D,Q)$, and
  - the probability that it's not relevant, $P(\neg R|D,Q)$

- To rank documents, we are interested in the **Odds of Relevance**, which is the ratio of the probability of relevance to the probability of non-relevance

- Using Bayes' Theorem

$$P(R|D,Q) = \frac{P(D|R,Q)P(R \mid Q)}{P(D \mid Q)}$$

and similarly for non-relevance

$$P(\neg R|D,Q) = \frac{P(D|\neg R,Q)P(\neg R \mid Q)}{P(D \mid Q)}$$

# Probabilistic Model of Relevance

- **Odds of Relevance**

$$Odds(R|D,Q) = \frac{P(R|D,Q)}{P(\neg R|D,Q)} = \frac{P(D|R,Q)}{P(D|\neg R,Q)} \frac{P(R \mid Q)}{P(\neg R \mid Q)}$$

- The **last term is constant for all documents** for a given query and can be ignored for ranking purposes (as it doesn't change the relative order of documents).

- So, we focus on the **Retrieval Status Value (RSV)**:

$$RSV(D,Q) = \frac{P(D|R,Q)}{P(D|\neg R,Q)}$$

- RSV gives the likelihood of observing document *D* if it is relevant to query *Q* divided by the likelihood of observing *D* if it is not relevant.
  - RSV > 1 : evidence favours relevance
  - RSV < 1 : evidence favours non-relevance.
  - Larger RSV : stronger evidence that the document should rank highly.

# Probabilistic Model of Relevance

$$RSV(D, Q) = \frac{P(D|R, Q)}{P(D|\neg R, Q)}$$

- Considering the binary representation and term independence.
  - $D = (d_1, d_2, \ldots, d_N)$ where $d_i = 1$ if term $i$ is in $D$ and $d_i = 0$ otherwise
  - $Q = (q_1, q_2, \ldots, q_N)$ where $q_i = 1$ if term $i$ is in $Q$ and $q_i = 0$ otherwise.

  Note: *D* and *Q* represent a specific document and query, respectively.

- Due to term independence, $P(D|R, Q)$ can be broken down into the product of individual term probabilities:

$$P(D|R, Q) = \prod_{i=1}^{n} P(d_i|R, Q)$$

and

$$P(D|\neg R, Q) = \prod_{i=1}^{n} P(d_i|\neg R, Q)$$

Substituting these back into the RSV equation, and simplifying for terms present in the query

$$RSV(D, Q) = \frac{\prod_{i=1}^{n} P(d_i|R, Q)}{\prod_{i=1}^{n} P(d_i|\neg R, Q)}$$

# Probabilistic Model of Relevance

| | |
|---|---|
| $p_i = P(d_i = 1 \mid R, Q)$ | **Probability the term appears in relevant docs.**<br>The probability that term $i$ occurs in a document given that the document is relevant to query $Q$.<br>Estimated from the proportion of relevant documents containing term $i$. |
| $u_i = P(d_i = 1 \mid \neg R, Q)$ | **Probability the term appears in non-relevant docs.**<br>The probability that term $i$ occurs in a document given that the document is non-relevant to query $Q$.<br>Estimated from the proportion of non-relevant documents containing term $i$. |

Under term independence and Bernoulli terms:

$$P(D, \mid R, Q) = \prod_{i=1}^{N} p_i^{d_i} (1 - p_i)^{1-d_i}$$

$$P(D, \mid \neg R, Q) = \prod_{i=1}^{N} u_i^{d_i} (1 - u_i)^{1-d_i}$$

$$\log \frac{P(D, \mid R, Q)}{P(D, \mid \neg R, Q)} = \sum_{i=1}^{N} \left[ d_i \log \frac{p_i}{u_i} + (1 - d_i) \log \frac{1 - p_i}{1 - u_i} \right]$$

Note: because $d_i$ is binary, the terms on the right will just yield $p_i$ or $1 - p_i$

# Probabilistic Models of Relevance

- Only query terms ($q_i = 1$) are scored. Terms with $q_i = 0$ do not contribute in practice.
- For $q_i = 1$:

$$RSV(D, Q) = \sum_{i:q_i=1} d_i \underbrace{\log \frac{p_i(1-u_i)}{u_i(1-p_i)}}_{w_i} + \underbrace{\sum_{i:q_i=1} \log \frac{1-p_i}{1-u_i}}_{\text{const. w.r.t. } D}.$$

- The second sum is constant across documents for a fixed query (only depends on query terms), so it can be dropped for ranking purposes.

$$\text{score}(D, Q) = \sum_{i:\ q_i=1} d_i \log\left(\frac{p_i(1-u_i)}{u_i(1-p_i)}\right)$$

- Equivalently,

$$\text{score}(D, Q) = \sum_{i \in Q \cap D} w_i \qquad \text{where} \qquad w_i = \log\left(\frac{p_i(1-u_i)}{u_i(1-p_i)}\right)$$

- Estimating $p_i, u_i$ from judged relevant/non-relevant sets yields the **binary independence model (BIM)** score.
- **Best Matching 25 (BM25)** is a refined practical variant that further takes into account term-frequency saturation and document-length normalisation.

# Relevance Feedback in the Probabilistic Model

- Initially $p_i$ and $u_i$ are not known, because we don't have any insight on which document are relevant or not.
  - Initially these terms have to be approximated.

| Term | Definition | Initial Approximation | Rationale |
|---|---|---|---|
| $u_i$ | $P(\text{term } i \mid \text{non}-\text{relevant})$ | $\frac{df_i}{N}$<br><br>$df_i$ = number of documents in the collection that contain term $i$ at least once.<br><br>$N$= number of documents. | Use overall document frequency as a proxy for how often the term occurs in "random" documents (initially assumed non-relevant).<br><br>Assumes that the average document is non-relevant. (Relevant ones are rare). |
| $p_i$ | $P(\text{term } i \mid \text{relevant})$ | Set heuristically (e.g. 0.5 for all query terms, or slightly higher for rare terms) | At the start we have no relevance labels, so we assume query terms are somewhat discriminative.<br><br>Rare terms may get a slightly higher prior probability, because they are generally stronger indicators of relevance. |

- Once the user (or system, via pseudo-feedback) provides relevance labels for the documents, data-driven values of $p_i$ and $u_i$ can be estimated.

| Case | $p_t = P(t \mid R = 1)$ | $u_t = P(t \mid R = 0)$ | Interpretation | Effect on Retrieval |
|---|---|---|---|---|
| **High $p_t$ , Low $u_t$** | Term often appears in relevant docs | Term rarely appears in non-relevant docs | Strong discriminator of relevance | Term weight should be **increased** (pushes ranking toward docs containing it) |
| **Low $p_t$ , High $u_t$** | Term rarely appears in relevant docs | Term often appears in non-relevant docs | Indicator of non-relevance | Term weight should be **decreased** (or even negative) |
| $p_t \approx u_t$ | Term appears with similar frequency in relevant and non-relevant docs | Term appears with similar frequency in relevant and non-relevant docs | Weak signal, little discriminative power | Term weight contributes **little or nothing** to ranking |

# Probabilistic Retrieval Framework

- **Without feedback**
  - The system must rely only on collection statistics (e.g., term frequency, document frequency) to approximate $P(R, = 1 \mid q, d)$.

- **With feedback**
  - Judgments from $D_r$ and $D_{nr}$ act as **samples of relevance**, providing direct evidence to update term weights.
  - If a term frequently appears in relevant documents but rarely in non-relevant ones, it becomes more strongly weighted.
  - Conversely, terms common in non-relevant documents are down-weighted.

# Using Feedback

**Smoothing to avoid extreme values**

- Without smoothing:

$$w_t = \log \frac{p_t(1-u_t)}{u_t(1-p_t)}$$

  - If a term occurs in **all relevant docs**, $\hat{p}_i = 1$.
  - If a term occurs in **no relevant docs**, $\hat{p}_i = 0$.

- Both cases cause problems in the weight formula:
  - If $\hat{p}_i = 1$ denominator has $(1-\hat{p}_i) = 0$
    - infinite weight.
  - If $\hat{p}_i = 0$ numerator has $\hat{p}_i = 0$
    - weight approaches $-\infty$.

- Similar issues if $\hat{u}_i = 0$ or $1$.

**With smoothing**:

$$\hat{p}_i = \frac{|\, R_i \,| + 0.5}{|\, R \,| + 1}$$

$$\hat{u}_i = \frac{|\, NR_i \,| + 0.5}{|\, NR \,| + 1}$$

Adding **0.5 pseudo-counts** prevents these extreme values:

$|\, R_i \,| + 0.5$ :treat the count as if we had seen “half an occurrence”
$|\, R \,| + 1$: treat the sample size as if it had one extra document

This “softens” the estimate so it’s never 0 or 1, keeping weights finite, allowing for very high weights that however do not tend to infinity.

# Relevance Feedback in the Probabilistic Model: Example

| Doc/Terms | diabetes | insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |

**Initial Approximation Before Feedback**

$$u_i \approx \frac{df_i}{N}, \qquad p_i = 0.5 \text{ } \textit{(heuristic)}$$

**Term 'diabetes':** $u = 3/5 = 0.6$ $p = 0.5$

**Term 'insulin':** $u = 2/5 = 0.4$ $p = 0.5$

**Weights** $w_i = \log \frac{p_i(1-u_i)}{u_i(1-p_i)}$

**Term 'diabetes':** $\log \frac{0.5(0.4)}{0.6(0.5)} = \log(0.667) = -0.18$

**Term 'insulin':** $\log \frac{0.5(0.6)}{0.4(0.5)} = \log(1.5) = 0.41$

**Initial contribution**: *diabetes* slightly negative, *insulin* positive.

# Relevance Feedback in the Probabilistic Model: Example

$$\text{score}(D, Q) = \sum_{i \in Q \cap D} w_i$$

| Doc/Terms | diabetes | insulin |
|---|---|---|
| **D1** | 1 | 1 |
| **D2** | 0 | 0 |
| **D3** | 1 | 0 |
| **D4** | 0 | 1 |
| **D5** | 1 | 0 |

- D1: (-0.18+0.41)=0.23
- D3: (-0.18)
- D4: (+0.41)
- D5: (-0.18)
- D2: 0

**With User Feedback**: Relevant = {D1, D3}, Non-relevant = {D4, D5}.

Re-estimate:

$$\hat{p}_i = \frac{| R_i | + 0.5}{| R | + 1}$$

$$\hat{u}_i = \frac{| NR_i | + 0.5}{| NR | + 1}$$

# Relevance Feedback in the Probabilistic Model: Example

**Term 'diabetes'**:

- Relevant: appears in both 2 out of 2: $\hat{p} = (2 + 0.5)/(2 + 1) = 0.833$
- Non-relevant: appears in 1 out of 2: $\hat{u} = (1 + 0.5)/(2 + 1) = 0.5$
- New weight: $\log \frac{0.833(0.5)}{0.5(0.167)} = \log(5) = 1.61$

**Term 'insulin'**:

- Relevant: appears in 1 out of 2: $\hat{p} = (1 + 0.5)/3 = 0.5$
- Non-relevant: appears in 1 out of 2 $\hat{u} = (1 + 0.5)/3 = 0.5$
- New weight: $\log \frac{0.5(0.5)}{0.5(0.5)} = \log(1) = 0$

Updated contribution: *diabetes* strongly positive (1.61), *insulin* neutral (0)

$$\text{score}(D, Q) = \sum_{i \in Q \cap D} w_i$$

Updated ranking

D1: 1.61
D3: 1.61
D5: 1.61
D4: 0
D2: 0

- **Limitations of the Probabilistic Model (BIM)**
  - **Binary Representation**: Ignores term frequency within documents, which is a strong indicator of importance.
  - **Term Independence**: This assumption is rarely true. Terms are often correlated (e.g., "artificial" and "intelligence").
  - **Difficulty in Estimating** $p_t$: Requires known relevant documents, which are often scarce without user feedback.
- Despite its limitations, the Binary Independence Model provides a strong theoretical foundation for probabilistic ranking.
- Laid the groundwork for more advanced models like BM25, which addresses some of its shortcomings by incorporating term frequency and document length.
- BM25 remains widely used in search and is often combined with vector retrieval in modern RAG systems.

| Failure Case | Example | Challenge | Probabilistic Alternative |
|---|---|---|---|
| **Synonymy (multiple vocabularies for same concept)** | Burma vs. Myanmar | Relevant docs split into two vocab clusters. Rocchio centroid falls in between, not close to either. | Each synonym term gets boosted if its $p_t \gg u_t$ ,so both Burma and Myanmar rise in weight. |
| **Disjunctive Queries** | "Pop stars who once worked at Burger King" | Relevant docs cover multiple disjoint clusters (different celebrities). Rocchio centroid averages them, pointing nowhere useful. | Each term (celebrity name) can be weighted independently by discriminative power. |
| **General Concept with Specific Subclusters** | Felines: lions, tigers, domestic cats | Relevant docs form several subclusters. Centroid compromises by over-favoring common words ("animal"), under-weighting distinctive species. | Each species term (lion, tiger, cat) can be promoted as long as it discriminates against nonrelevant docs. |
| **Heterogeneous Nonrelevant Documents** | Query: "Apple Inc. revenue" irrelevant docs about apple fruit, apple pies, gardening | Nonrelevant docs are scattered everywhere; centroid subtraction suppresses useful overlapping terms (e.g., "apple"). | Nonrelevant evidence used term-by-term; overlap terms can still survive if they discriminate relevance. |

# 4. Types of Feedback

# Explicit Relevance Feedback

| Type | Definition | Example | Key Point |
|---|---|---|---|
| **Binary Relevance Judgments** | User labels docs as *relevant / not relevant*. | Relevant / Not Relevant | Simple and quick, but does not capture degrees of relevance. |
| **Graded Relevance Judgments** | User assigns a level of relevance level. | 0 = irrelevant, 1 = somewhat relevant, 2 = useful, 3 = highly relevant. | More informative, but requires greater user effort. |
| **Natural-Language Feedback / Critiquing** | User explains why a result does or does not meet their need. | “Too general”, “too old”, “wrong domain”. | Can refine the information need beyond simple relevance labels; particularly useful in conversational systems. |
| **Example-Based Feedback** | User provides exemplar documents instead of verbal feedback. | Legal/patent search: “find more docs like this one.” | Intuitive and useful when the desired information is difficult to describe explicitly. |

- Modern systems can therefore obtain explicit relevance feedback through labels, ratings, conversational corrections, or examples.

# Explicit Relevance Feedback – Strengths and Limitations

| Aspect | Strength | Limitation |
|---|---|---|
| **Signal Quality** | Direct feedback provides a strong indication of what the user considers relevant. | Requires deliberate user effort, so it is rarely available at large scale. |
| **User Intent** | Can capture needs that are not obvious from the original query alone. | Judgements may still be subjective or inconsistent. |
| **Training and Evaluation** | Provides high-quality labels for evaluating and training information retrieval/ranking systems. | Expensive and labour-intensive to collect. |
| **Use in Practice** | Particularly useful in professional, specialist and interactive systems. | Usually combined with implicit or automatic feedback in large-scale systems. |

- In conversational/LLM systems, explicit feedback can also take the form of natural-language corrections, preferences or follow-up instructions.

# Implicit Relevance Feedback

| Category | Signal Type | Definition | Strengths | Limitations / Biases |
|---|---|---|---|---|
| **Click-based signals** | **Clickthrough** | User clicks a retrieved result, suggesting interest or possible relevance. | - Simple, abundant<br>- Easy to collect at scale | - Presentation/position bias (higher-ranked results get more clicks regardless of quality).<br>- Attractive snippets may inflate clicks without true relevance. |
| | **Click position** | Rank position at which user clicks (e.g., 1st vs 5th result). | - Used to correct for rank bias. | - Not itself a direct relevance signal (mainly useful for debiasing)<br>- Top results dominate clicks. |

# Implicit Relevance Feedback

| Category | Signal Type | Definition | Strengths | Limitations / Biases |
|---|---|---|---|---|
| **Time-based signals** | **Dwell time** | Time spent viewing a clicked document. | - Good indicator of user engagement and satisfaction | - Task-dependent (user may skim or get answer instantly).<br>- Background tabbing inflates times. |
| **Interaction signals** | **Scroll depth / engagement** | How much of the result the user reads or interacts with. | - Richer behavioural evidence than clicks alone. | - May reflect curiosity rather than relevance.<br>- Varies across interfaces/devices. |
| | **Saving / bookmarking / downloading / sharing** | User performs a stronger action on a result. | - Often stronger evidence of usefulness than just a click. | - Rare compared to clicks.<br>- Strongly domain-dependent. |

# Implicit Relevance Feedback

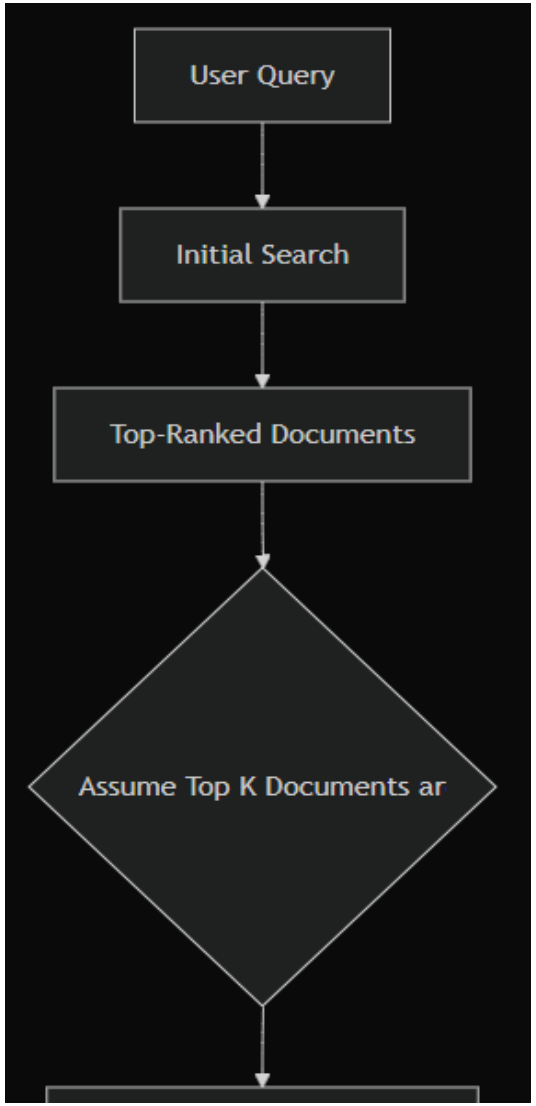
| Category | Signal Type | Definition | Strengths | Limitations / Biases |
|---|---|---|---|---|
| **Query-level signals** | **Query reformulation / follow-up** | User modifies query (e.g., “cat food” → “buy cat food online”). | - Reveals evolving intent and possible dissatisfaction with previous results.<br>- Particularly relevant in conversational search. | - Ambiguity: reformulation may be task-driven rather than due to poor results. |
| | **Abandonment / no-click** | User issues a query but selects no result (abandons session). | - Negative signal - results may not satisfy intent. | - Could also mean user found the answer directly from a snippet. |
| **Document-level signals** | **Repeat visits** | Returning to the same document later. | - Stronger evidence of long-term usefulness. | - May be influenced by external factors (e.g., saved links, shared in chat). |
| | **Citation / download logs** | User downloads, cites, exports, or otherwise reuses a result. | - Strong signal in scholarly and enterprise settings. | - Rare and often delayed. |

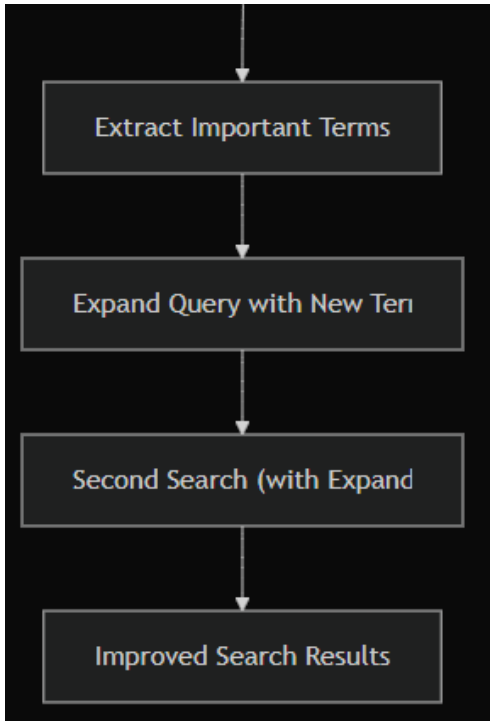
# Pseudo-Relevance Feedback

- **Pseudo-Relevance Feedback (PRF)**, also known as **blind relevance feedback**
  - used to improve initial query results without requiring explicit input from the user

- **Process**
  - **Initial Retrieval:** When a user submits a query, the system first performs an initial search and retrieves a set of top-ranked documents.
  - **Assumption of Relevance:** PRF then *assumes* that the top few documents (e.g., the top 10 or 20) from this initial retrieval are relevant to the user's query. This is **the "pseudo" part** - the system is treating these documents *as if* they were explicitly marked as relevant.

- The same principle is used in modern retrieval systems, although feedback may be extracted from **terms, embeddings or LLM-based analysis** of the top-ranked results.



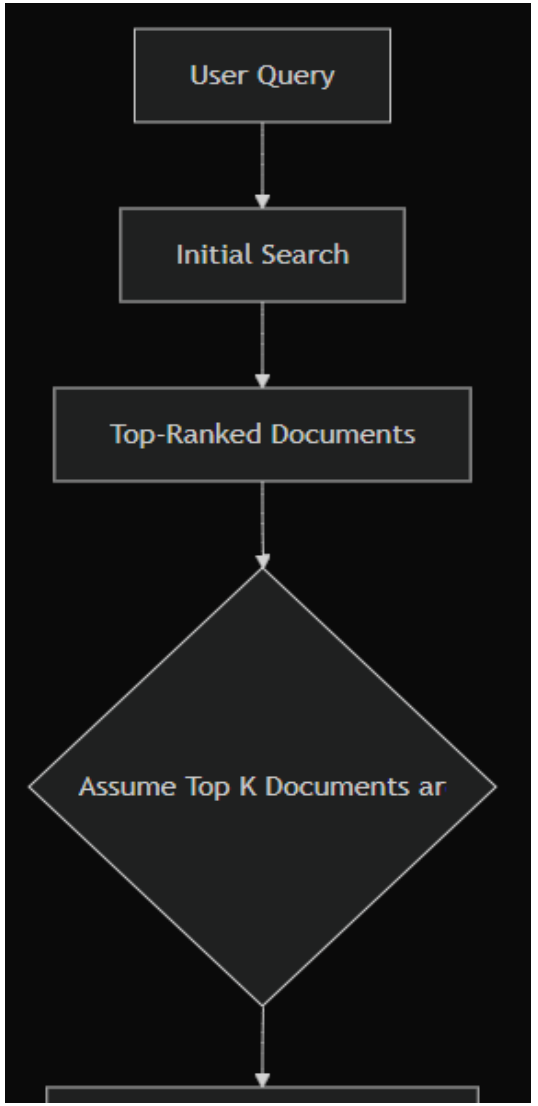



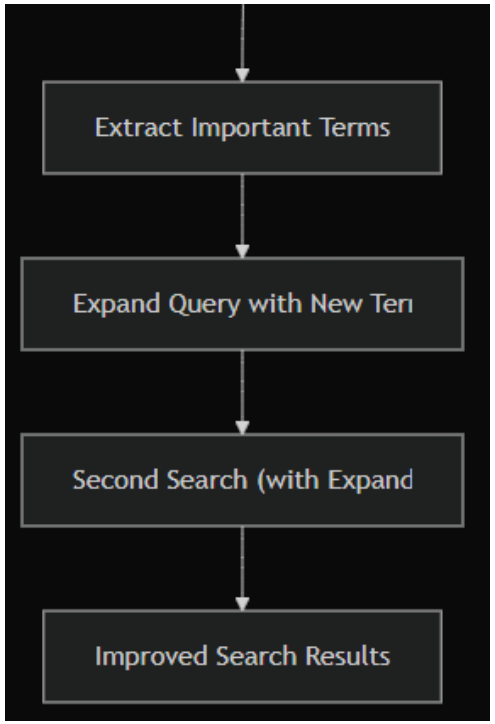
# Pseudo-Relevance Feedback

- **Term Extraction and Expansion:**
  - The system extracts useful information from the top-ranked documents.
  - Classical approaches identify and weight important **terms**.
  - Modern approaches may also use **semantic embeddings or LLMs** to identify useful concepts and context.

- **Query Expansion:**
  - The original query is modified using this information.
  - Classical example: **Rocchio-based expansion and term reweighting**.
  - Modern approaches may **rewrite the query, generate alternative queries or modify its embedding representation**.

- **Re-ranking:** The system then performs a second retrieval using this expanded query, which often leads to more accurate and relevant results being ranked higher.



# Pseudo-Relevance Feedback - Pros and Cons

- **Advantages**
  - **Automation:** It's fully automatic, requiring no extra effort from the user.
  - **Query Refinement:** It helps in cases where the initial query is too short, ambiguous, or doesn't fully capture the user's intent.
  - **Vocabulary/Semantic Mismatch:** It can alleviate issues of vocabulary mismatch between the user's query and the terms used in relevant documents
- **Limitations**
  - **Dependence on Initial Retrieval:** PRF assumes that the top-ranked documents are useful. If the initial ranking is poor, misleading evidence may be used.
  - **Query Drift:** PRF can lead to "query drift," where the expanded query moves away from the user's true information need, potentially worsening the results.
  - **Parameter Tuning:** The number of documents to assume as relevant and how many terms to extract are crucial parameters that can require careful tuning.

| Aspect | Explicit Relevance Feedback | Pseudo Relevance Feedback (Blind) | Implicit Relevance Feedback |
|---|---|---|---|
| **User involvement** | User explicitly marks documents as relevant or non-relevant. | System automatically assumes top-k returned documents are relevant. | System infers feedback indirectly from user behaviour (clicks, dwell time, query reformulation). |
| **Signal Reliability** | High – based on actual user judgements. | Variable – risk of error (query drift) if top-k are not truly relevant. | Moderate – signals are noisy (click ≠ relevance), but good at scale. |
| **Main drawback** | Users reluctant to provide feedback; slows interaction. | Incorrect assumptions can push query in wrong direction (topic drift). | Behavioural signals are ambiguous (e.g. clickbait, position bias, accidental clicks). |
| **Use in practice** | Evaluation, specialist search, interactive/conversational systems and direct user corrections. Rare in web search. | Automatic query expansion and iterative retrieval; modern variants may use embeddings or LLMs. | Widely used in large-scale search, recommendation and ranking systems. |

# Real-World Examples of Relevance Feedback

| Domain / System | Type | Example | Notes |
|---|---|---|---|
| **Web Search / Recommendation (e.g. Google, YouTube, Amazon)** | Implicit | Clicks and other interaction signals provide evidence about user preferences. | Scales to very large numbers of users, although signals are noisy and biased. |
| **Image / Multimodal Retrieval (e.g. Google Lens)** | Explicit / Example-based | User selects an image and asks for “more like this”. | Useful when the desired content is difficult to describe precisely in words. |
| **Patent / Legal Search (e.g. Google Patents)** | Explicit + Pseudo | Relevant documents are used to refine technical terminology and retrieve related material. | High recall is important; helps overcome vocabulary mismatch. |
| **Scholarly / Biomedical Search (e.g. PubMed)** | Example-based + Pseudo | Relevant papers or initially retrieved papers can be used to find related literature and refine subsequent searches. | Supports exploratory search across specialised terminology. |
| **Conversational / RAG Systems (e.g. ChatGPT, Copilot)** | Explicit + Pseudo | User corrections or follow-ups refine subsequent retrieval; top-ranked passages may be used to reformulate or repeat the search. | Improves retrieval as the system gains more evidence about the user’s intent. |

# 4. Query Expansion Methods

# Query Expansion Methods

- **Motivation**
  - **Vocabulary mismatch:** users and relevant documents may express the same concept using different terms.
    *Example: “cancer” vs “neoplasm”; “aircraft” vs “plane”.*
  - **Ambiguity:** short queries may have multiple interpretations.
    *Example: “Jaguar speed” could mean speed of the car or of the animal.*
  - **Incomplete queries:** users often omit useful context or terminology.
    *Example: “diabetes treatment”*
    *Important context such as type, population, recency and treatment modality is missing.*
    *User may actually mean: “latest drug treatments for type 2 diabetes in adults”*
- **Goal:** modify the query so that it better represents the underlying information need.

# Query Expansion Methods

At a high level, query expansion typically involves these steps:

- **Initial Query Formulation:** The user provides an initial query (e.g., "fast cars").
- **Suggestion/Selection:** The system identifies potentially useful terms, concepts or alternative formulations using methods such as relevance feedback, thesauri, embeddings or LLMs.
- **Query Modification:** The original query is modified by adding, removing, adding/re-weighting terms, rewriting the query, or generating alternative queries..
- **Re-execution of Query:** The expanded query is then used to search the document collection.
- **Result Evaluation:** The new results are presented to the user, and the effectiveness of the expansion can be evaluated, with potential further refinement..

# Local Methods

- **Local methods use feedback from the *retrieved set* (top-k documents)**
  - **Rocchio algorithm**: adjusts the query vector towards relevant documents and away from non-relevant ones.
  - **Probabilistic feedback**: reweights terms according to how strongly they distinguish relevant from non-relevant documents.
  - **Pseudo-Relevance Feedback (PRF)**: assumes the top-k retrieved documents are relevant and uses them to refine the query automatically.
  - **Modern variants**: retrieved passages may also be used to refine query embeddings or generate improved query rewrites with an LLM

- **Strengths**
  - Adapts dynamically to the query and collection
  - Often boosts recall.

- **Weaknesses**
  - Highly dependent on first-pass ranking quality
  - Query drift if top-k contains noise
  - Requires parameters – how many documents/terms to consider

# Global Methods

- Global query expansion methods use global, collection-independent knowledge to find synonyms, related terms, or semantic associations.

**1. Manual thesauri**

- Controlled vocabularies built by domain experts; provide precise synonyms and hierarchical relations.
- Controlled vocab: Domain experts create controlled vocabularies (e.g. MeSH for biomedical, EuroVoc for legal).
- Example: Query: *“heart attack”* expanded to *“myocardial infarction”*.
- **Specialist domains only** (PubMed, legal databases, patents).
- Not feasible for general web search (too narrow, labour-intensive).

# Global Methods

## 2. Automatic thesauri

- Expansion terms are derived automatically by analysing large text collections.
- The system identifies words that **frequently co-occur** with a query term, or that appear in **similar contexts**.
- Embedding-based methods generalise this idea by learning semantic relationships from context rather than relying only on explicit co-occurrence counts.
- This creates a “statistical thesaurus” without the need for human curation.

- **Example:**
  - Query: *“climate change”* - *f*requent co-occurrence patterns suggest related terms like: *“global warming”, “greenhouse gases”, “carbon emissions”*

- **Obtained through:**
  - Term co-occurrence matrices: count how often two terms appear together in the same document, paragraph, or window.
  - Association measures (e.g. Pointwise mutual Information, Dice coefficient) identify strong term–term links.
  - Distributional similarity: if two words appear in similar contexts across the corpus, they are considered semantically related.

# Global Methods

**3. Knowledge-based resources**

- Use **structured lexical or encyclopaedic resources** to suggest synonyms, hypernyms, hyponyms, or related concepts.
- These are **curated knowledge bases** that encode explicit semantic relationships between terms.

- For example
  - **WordNet** (lexical database): Provides synonym sets ("synsets"), hypernyms (broader terms), hyponyms (narrower terms).
  - **DBpedia** (structured from Wikipedia infoboxes): Entity-centric: links terms to entities and related attributes.

- **Domain ontologies** (e.g. UMLS in medicine, EuroVoc in law): Capture controlled terminology and relationships in specialist areas.
  - Example:
    - Query: *"dog"* → *"canine, domestic dog, pooch"* (WordNet).
      Query: *"France"* → *"Paris, EU member state"* (DBpedia).

  - Provide high-precision synonyms and well-defined relations. Useful in specialist domains where terminology is consistent and curated.
  - Often too small or static for open-domain web search.

# Global Methods

**4. Embeddings/Semantic Methods**

- Embeddings represent words, queries or documents as dense numerical vectors in which semantic similarity is reflected by proximity.
- Unlike classical VSM vectors, the dimensions are learned semantic features, rather than individual vocabulary terms.

- **Techniques**
  - **Word Embeddings (e.g. Word2Vec)**
    - Train embeddings over a large corpus.
    - Expansion terms are selected from the nearest neighbours in embedding space.
  - **Sentence / Query / Document Embeddings**
    - Instead of word-level, compute embeddings for the whole query.
    - Retrieval can then identify semantically similar documents without explicitly adding new query terms.
  - **Hybrid (Embeddings + PRF)**
    - Pseudo-relevance feedback generates candidate expansion terms.
    - Embeddings help identify and retain semantically relevant candidates.
    - This can reduce noisy expansion and limit query drift.

# 5. Applications of RF and QE

# Applications of RF & QE

| Application area | RF / QE use |
|---|---|
| **Web Search** | Implicit feedback, query reformulation, spelling/synonym expansion and semantic retrieval improve ranking. |
| **Conversational / RAG Systems** | Follow-up queries, corrections and retrieved evidence refine subsequent retrieval. |
| **Legal / Patent Search** | QE handles terminology variants; high recall is often critical. |
| **Biomedical / Clinical IR** | MeSH/UMLS and related vocabularies reduce terminology mismatch. |
| **E-commerce** | Clicks, purchases and semantic matching improve product retrieval. |
| **Scholarly Search** | Related-paper retrieval, PRF and semantic search support exploration. |

Classical relevance feedback and query expansion remain relevant, but modern systems increasingly combine them with embeddings, hybrid retrieval and LLM-based query reformulation.